

Orycterope Transalpin IT test

Assembling the workstation

Inserting / Changing RAM

1. Open the side panel with the lever
2. Remove the CPU dust protection



3. Insert ram sticks while following the RAM order from the motherboard

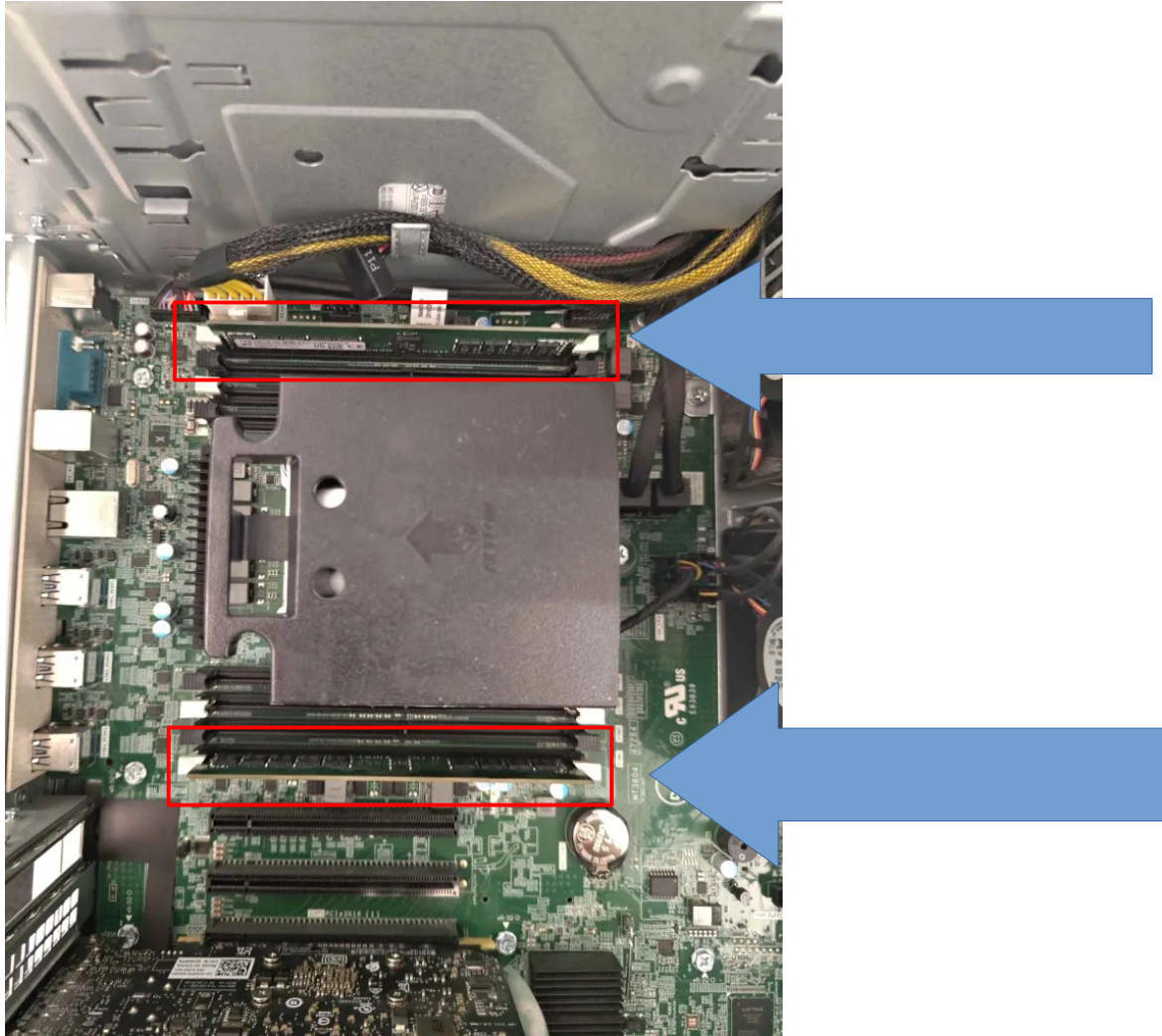
Memory Load Order

1
5
3
7



8
4
6
2

Result



Inserting / Changing Graphic card

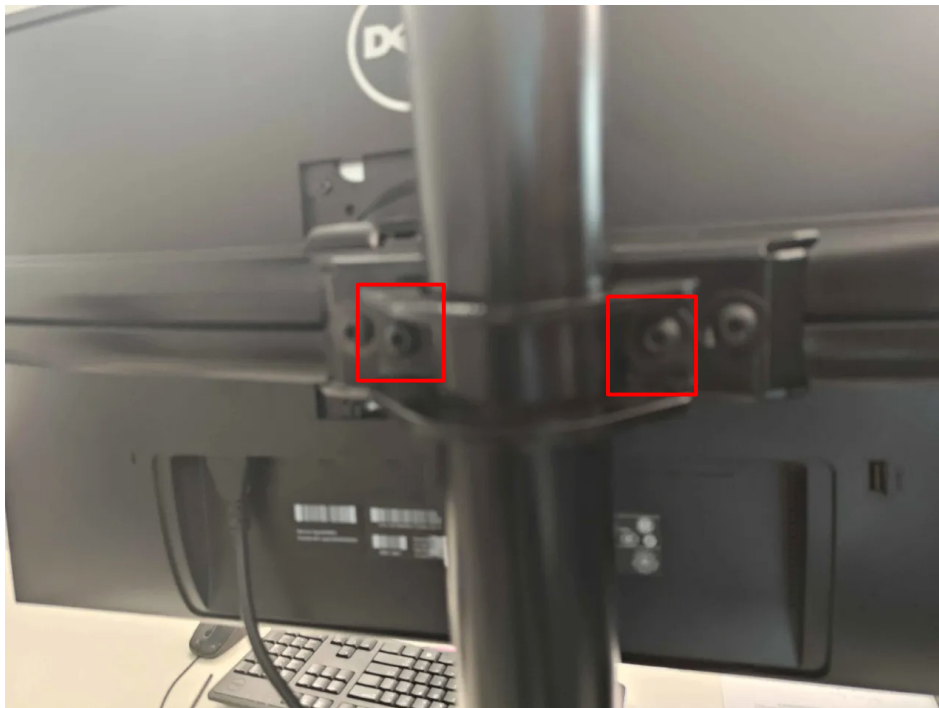
1. Open the locks on the side (blue switches)
2. Remove one of the PCI-e cover
3. Insert Graphic card

Result

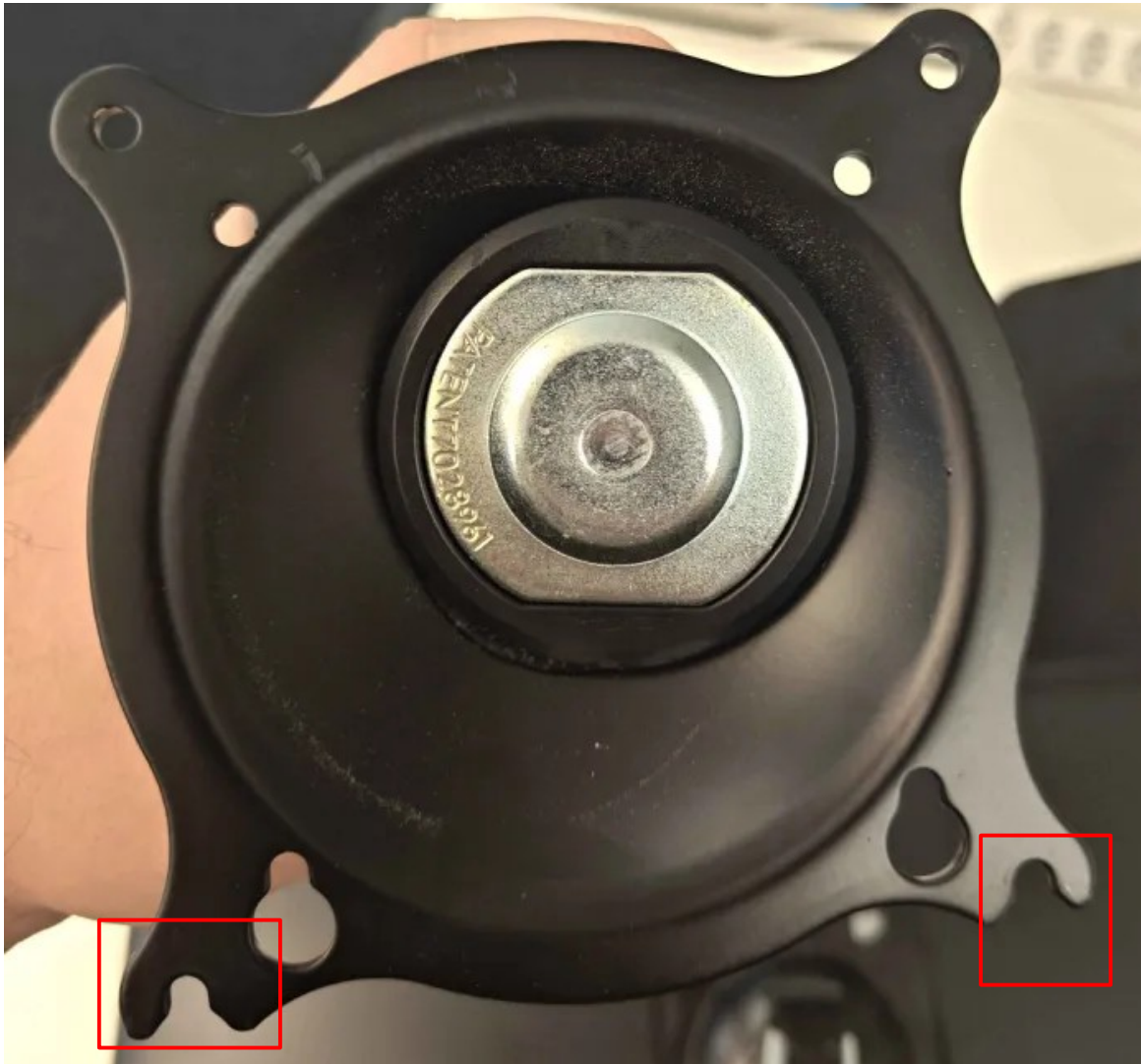


Mount the stand

If the horizontal bar is off the stand, you need to unscrew a little bit both of the screws that are holding the rings.



Once the horizontal bar is fixed up, we can screw the screen holder. Beware that it is in the right orientation, there is 2 screws that isn't full rounded and it may become source of potential fall of the screen !



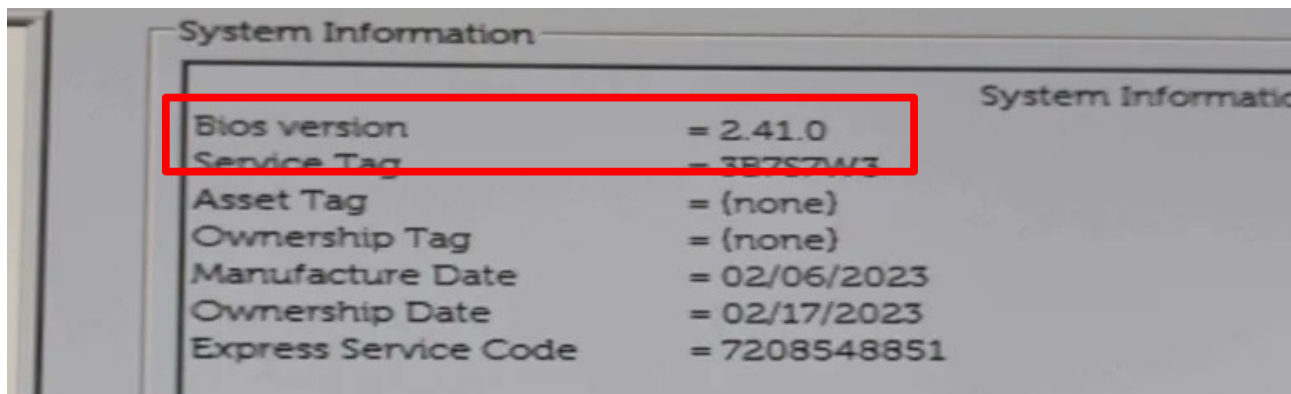
Theses one to be on the bottom of the screen. Like that.



Checking the BIOS version

To check the BIOS version, you first need to enter the BIOS. Reboot the computer and when you see the DELL Logo, press F2. You'll enter the BIOS and then you can go in Settings → General → System information.

You'll then be able to see the BIOS version.



Install the OS

- Plug the USB disk (in a USB3.0 port if possible for faster installation)
- Reboot the computer
- At the Dell LOGO press F12
- Select the Sandisk Extreme disk (USB key)
- There was a lot of ISO available, checked with Artur and installed the ubuntu 24.04 iso
- You're prompted what you want to do, select "Try or Install Ubuntu"
- The computer reboot and arrives on the Ubuntu desktop.
- If the installer isn't starting there is a shortcut on the Desktop.
- Follow the instructions
- For the local user, I've created
 - Username : lorenzo
 - Password : admin123
- Once the installation has started, it take some times (~5-10 minutes)
- When it finished it will ask you to reboot, do it and before shutting down, it will ask you to remove the usb key.

What is ansible

Ansible is a automatization tool for linux, that allows you to manage a whole IT parc. You can use inventories, groups of workstations or even target one workstation, and make them run what we call a playbook. A playbook is basically a list of tasks you'll ask the remote workstations to do.

Install ansible

- Open a terminal
- Type the following commands

```
sudo apt update
sudo apt install software-properties-common
sudo add-apt-repository --yes --update ppa:ansible/ansible
sudo apt install ansible
```

We always start by updating apt, to be sure we retrieve the latest versions of packages we want to install.

Then we install a common package used by ansible. It will mostly be already installed but it doesn't hurt to check first.

After that we update the apt's repository, so we can add the ansible repos. And then, we can finally install apt.

- It will prompt for the root password, type it
- It will ask you 2-3 times if you're sure you want to install, type yes

Creating the playbook to install Docker

- First, we need Ansible to be installed.
- Create a folder (on the Desktop for me)
- In this folder, we will create 2 files. One for the inventory and the other one for the playbook.
- The inventory file is used to let you manage multiple workstations at once.
 - It can be either a .ini file or a .yaml file, but ansible recommends using .ini file for the scalability.
 - Since it is a test scenario, we will create only one group of workstations and set only our local workstations
 - Here is the content of the file.



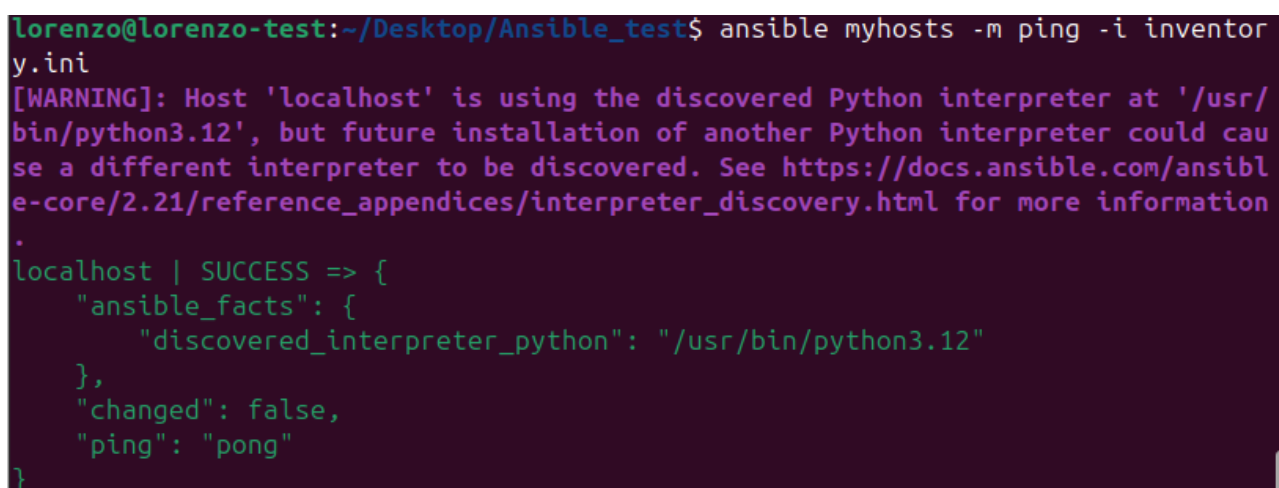
```
Open ▾ [🔍] inventory.ini
~/Desktop/Ansible_test

[myhosts]
localhost ansible_connection=local
```

- You can check if your inventory works by opening a terminal and typing this command

```
ansible myhosts -m ping -i inventory.ini
```

- It should return you a success like this.



```
lorenzo@lorenzo-test:~/Desktop/Ansible_test$ ansible myhosts -m ping -i inventory.ini
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.21/reference_appendices/interpreter_discovery.html for more information
.
localhost | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}
```

- The playbook, is a series of task you will ask Ansible to perform.
 - You need to create a .yaml file (in my case it's docker_playbook.yaml)
 - The file is segmented in different sections.
 - Global configurations

```
---
- name: Install docker playbook
  hosts: myhosts
  become: true
```

- Name : the name of the playbook
- hosts : the group of hosts to target from our inventory file (can be set to ALL)
- become : keep the privileges according (Will then ask to be performed as root)

- Tasks

```
tasks:
- name: Update apt cache
  ansible.builtin.apt:
    update_cache: yes
```

- Name : the name of the task
- Module : It is the module that ansible will use to perform your task. There is a large variety of module in the builtin library and you can probably download new modules from the community or develop your own.

Task to install docker

While there is many tasks in my playbook, theses ones are the most important to install docker.

First we need to create a directory to store the GPG of docker.

```
- name: Create GPG directory
  ansible.builtin.file:
    path: /etc/apt/keyrings
    state: directory
    mode: "0755"
```

- Path : Where we create the directory
- State : The type of file we create
- Mode : The permissions we give the file

Then we need to download Docker's GPG

```
- name: Download Docker's GPG
  ansible.builtin.get_url:
    url: https://download.docker.com/linux/ubuntu/gpg
    dest: /etc/apt/keyrings/docker.asc
    mode: "0644"
```

- url : The URL from which we will download a file
- dest : The destination folder
- mode : The permissions we give the file

After that come the addition of the Docker repository to our apt repository.

```
- name: Add docker repository to apt sources
  ansible.builtin.deb822_repository:
    name: docker
    types: deb
    uris: https://download.docker.com/linux/ubuntu
    suites: '{{ ansible_distribution_release }}'
    components: stable
    architectures: amd64
    signed_by: /etc/apt/keyrings/docker.asc
```

- Name : The name of the repository
- Types : The type of packages it contains
- Uris : One or many URL for APT to check for packages
- Suites : The release of our version of Ubuntu. (Here it is replaced by an ansible variable)
- components : The type of release we want.
- Architectures : The architecture of our workstation. (Can be replaced by an ansible variable, but it return me x86_64 and so doesn't work)
- signed_by : The path to the GPG we downloaded earlier

And finally, the installation task (after an apt update !)

```
- name: Install Docker-Engine
  ansible.builtin.apt:
    name:
      - docker-ce=5:26.0.0-1~ubuntu.24.04~noble
      - docker-ce-cli=5:26.0.0-1~ubuntu.24.04~noble
      - containerd.io
      - docker-buildx-plugin
      - docker-compose-plugin
    state: present
```

- name : The packages name we want to install. Can be single or multiples packages
- State : The state of the packages should be at the end of the task.

Result

The playbook did not run by himself the first time. See the problem encountered section at the end of the documentation. After the modifications, it did without errors.

```
lorenzo@lorenzo-test:~/Desktop/Ansible_test$ sudo ansible-playbook -i inventory.ini docker_playbook.yaml
```

```
PLAY [Install docker playbook] *****
```

```
TASK [Gathering Facts] *****
```

```
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.21/reference\_appendices/interpreter\_discovery.html for more information
```

```
ok: [localhost]
```

```
TASK [Update apt cache] *****
```

```
changed: [localhost]
```

```
TASK [Install certificate and curl] *****
```

```
ok: [localhost]
```

```
TASK [Create GPG directory] *****
```

```
ok: [localhost]
```

```
TASK [Download Docker's GPG] *****
```

```
ok: [localhost]
```

```
TASK [Add docker repository to apt sources] *****
```

```
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False` in ansible.cfg.
```

```
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to `True` is deprecated, top-level facts will not be auto injected after the change. This feature will be removed from ansible-core version 2.24.
```

```
Origin: /home/lorenzo/Desktop/Ansible_test/docker_playbook.yaml:35:17
```

```
33         types: deb
```

```
34         uris: https://download.docker.com/linux/ubuntu
```

```
35         suites: '{{ ansible_distribution_release }}'
```

```
^ column 17
```

```
Use `ansible_facts["fact_name"]` (no `ansible_` prefix) instead.
```

```
ok: [localhost]
```

```
TASK [Update apt cache again] *****
```

```
changed: [localhost]
```

```
TASK [Install Docker-Engine] *****
```

```
changed: [localhost]
```

```
TASK [Ensure Docker service is enabled and started] *****
```

```
ok: [localhost]
```

```
PLAY RECAP *****
```

```
localhost                : ok=9    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

As asked in the task, we can check the docker version by typing the command

```
docker --version
```

and see that we are in the correct version

```
lorenzo@lorenzo-test:~/Desktop/Ansible_test$ docker --version
Docker version 26.0.0, build 2ae903e
lorenzo@lorenzo-test:~/Desktop/Ansible_test$
```

Gather basic informations

Ports open

I used NMAP to check what ports are open on the workstation.

```
lorenzo@lorenzo-test:~/Desktop/Ansible_test$ nmap 127.0.0.1
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-20 10:58 CEST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000069s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT      STATE SERVICE
631/tcp   open ipp

Nmap done: 1 IP address (1 host up) scanned in 0.07 seconds
```

If we want to see all the connections the workstation is currently using, we can use lsof

```
lorenzo@lorenzo-test:~/Desktop/Ansible_test$ lsof -i
COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
Firefox 4241 lorenzo 13u IPv4 72189 0t0 TCP lorenzo-test.transmarketgroup.com:52388->a23-215-5-203.deploy.static.akamaitechnologies.com:https (ESTABLISHED)
Firefox 4241 lorenzo 42u IPv4 56503 0t0 TCP lorenzo-test.transmarketgroup.com:54932->162.159.133.234:https (ESTABLISHED)
Firefox 4241 lorenzo 94u IPv4 89404 0t0 TCP lorenzo-test.transmarketgroup.com:50284->93.243.107.34.bc.googleusercontent.com:https (ESTABLISHED)
Firefox 4241 lorenzo 98u IPv4 72192 0t0 UDP *:53192
Firefox 4241 lorenzo 108u IPv4 72198 0t0 UDP *:55814
Firefox 4241 lorenzo 135u IPv4 87385 0t0 UDP *:60364
Firefox 4241 lorenzo 229u IPv4 25079 0t0 TCP lorenzo-test.transmarketgroup.com:47294->39.224.186.35.bc.googleusercontent.com:https (ESTABLISHED)
```

Used and available disk space

To see available and used disk space, there is the command `df -h`

```
lorenzo@lorenzo-test:/$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           3.2G  2.8M  3.2G   1% /run
/dev/nvme0n1p2  937G   19G  871G   3% /
tmpfs           16G   4.0K   16G   1% /dev/shm
tmpfs           5.0M   12K   5.0M   1% /run/lock
efivarfs        512K   88K   420K  18% /sys/firmware/efi/efivars
/dev/nvme0n1p1  1.1G   6.2M   1.1G   1% /boot/efi
tmpfs           3.2G   5.8M   3.1G   1% /run/user/1000
```

Our main disk here is the `/dev/nvme0n1p2`. The `/dev/nvme0n1p1` is the boot volume.

Problem encountered

Docker version 25.0.5 not available on ubuntu 24.04

After my first run of the playbook, there was an error.

```
TASK [Install Docker-Engine] *****
[ERROR]: Task failed: Module failed: no available installation candidate for doc
ker-ce=5:25.0.5-1~ubuntu.22.04~jammy
Origin: /home/lorenzo/Desktop/Ansible_test/docker_playbook.yaml:44:7

42         update_cache: yes
43
44     - name: Install Docker-Engine
      ^ column 7

fatal: [localhost]: FAILED! => {"cache_update_time": 1787214270, "cache_updated"
: false, "changed": false, "msg": "no available installation candidate for docke
r-ce=5:25.0.5-1~ubuntu.22.04~jammy"}
```

This error occurred when Ansible tried to install Docker version 25.0.5 as asked. After checking manually, the version 25.0.5 is not available on Ubuntu 24.04 noble.

I noticed Artur on that fact and told me to install the first version available that is the 26.0.0.

```
docker-ce/noble 5:27.1.0-1~ubuntu.24.04~noble amd64
docker-ce/noble 5:27.0.3-1~ubuntu.24.04~noble amd64
docker-ce/noble 5:27.0.2-1~ubuntu.24.04~noble amd64
docker-ce/noble 5:27.0.1-1~ubuntu.24.04~noble amd64
docker-ce/noble 5:26.1.4-1~ubuntu.24.04~noble amd64
docker-ce/noble 5:26.1.3-1~ubuntu.24.04~noble amd64
docker-ce/noble 5:26.1.2-1~ubuntu.24.04~noble amd64
docker-ce/noble 5:26.1.1-1~ubuntu.24.04~noble amd64
docker-ce/noble 5:26.1.0-1~ubuntu.24.04~noble amd64
docker-ce/noble 5:26.0.2-1~ubuntu.24.04~noble amd64
docker-ce/noble 5:26.0.1-1~ubuntu.24.04~noble amd64
docker-ce/noble 5:26.0.0-1~ubuntu.24.04~noble amd64

lorenzo@lorenzo-test:~/Desktop/Ansible_test$
```

To see all the versions available, type the command

```
apt list --all-versions docker-ce
```

I then modified my playbook by replacing the version of docker to install

```
- name: Install Docker-Engine
  ansible.builtin.apt:
    name:
      - docker-ce=5:26.0.0-1~ubuntu.24.04~noble
      - docker-ce-cli=5:26.0.0-1~ubuntu.24.04~noble
```